Pencil Manufacturing Production Line Simulation

Course	Advanced Programming
Product	Pencil (graphite core, body, eraser, eraser holder)
Stack	Python · Flask · InfluxDB · Grafana · Docker

Introduction

1.1 Product Description

The chosen product is a **standard graphite pencil**, one of the world's most manufactured writing instruments. Despite its apparent simplicity, a pencil consists of four distinct sub-components that must be assembled in a precise sequence. The simulation models a realistic industrial production line that fabricates and assembles each component, applying probabilistic quality checks at every stage to replicate real-world defect rates.

1.2 Production Line Concept

The line is divided into four sequential manufacturing stages:

Stage 1 — Graphite Core Insertion

The graphite–clay mixture rod is pressed and fired, then inserted into a pre-routed cedar wood slot. Defect rate: 5 %. Failure mode: core fracture during insertion due to material brittleness.

Stage 2 — Pencil Body Assembly

Two halves of cedar wood are glued around the graphite core and clamped. The body is then shaped (round or hexagonal) and sanded. Defect rate: 4 %. Failure mode: body split caused by wood-grain defects or insufficient adhesive coverage.

Stage 3 — Eraser Attachment

A synthetic rubber eraser plug is inserted into the ferrule seat at the top end of the pencil body. Defect rate: 6 %. Failure mode: eraser misalignment or missing eraser due to feeder mechanism jam.

Stage 4 — Eraser Holder Crimping

A metal ferrule (eraser holder) is crimped around the eraser plug to secure it permanently. Defect rate: 3 %. Failure mode: crimp failure leading to a loose or detached ferrule.

1.3 Overall Quality Concept

A unit is marked **defective** as soon as a fault is detected at any stage — the line does not continue assembling a known-bad unit. The backend tracks each unit's status, the specific stage that failed, and the exact reason. All data is streamed to InfluxDB and displayed in real time via the HMI frontend and a Grafana dashboard.

1.4 Stage Overview Table

Stage	Component	Operation	Defect Rate	Failure Mode
1	Graphite Core	Insert & press	5 %	Core fracture
2	Pencil Body	Glue & shape	4 %	Wood-grain split
3	Eraser	Plug insertion	6 %	Misalign / missing
4	Eraser Holder	Crimp ferrule	3 %	Crimp failure

Task 2 — Program Implementation (60 Points)

2.1 System Architecture

The system follows a four-container Docker architecture, with each service fulfilling a dedicated role. All containers communicate over an internal Docker network, with only the required ports exposed to the host machine.

Container	Image / Tech	Port	Role
pencil_backend	Python 3.11 + Flask	5000	REST API — production logic
pencil_influxdb	InfluxDB 2.7	8086	Time-series metrics database
pencil_grafana	Grafana 10.2	3000	Live monitoring dashboard
pencil_hmi	Nginx + HTML/JS	80	Operator HMI frontend

Table 2 — Docker service architecture

2.2 Backend — Python / Flask

The backend is implemented in **app.py** using the Flask micro-framework and exposes four REST endpoints. The production loop runs on the frontend tick interval (1.2 s), keeping the backend stateless between requests and avoiding threading issues.

- POST /start — sets running=True, writes event to InfluxDB
- POST /stop — sets running=False, writes event to InfluxDB
- POST /reset — clears all counters and history, writes event to InfluxDB
- POST /tick — simulates one unit through all 4 stages; writes metrics
- GET /status — returns full machine state as JSON

Defect detection logic (per unit):

Each of the four stages is evaluated independently. A pseudo-random float is compared against the stage's defect rate constant. If the value falls below the threshold the unit is immediately flagged, the specific reason string is stored, and assembly stops for that unit. Stage temperatures are also simulated with bounded random walks to provide realistic sensor data.

Core defect simulation snippet:

```
for i in range(4):
    if random.random() < DEFECT_RATES[i]:
        return False, f"Stage {i+1}: {DEFECT_REASONS[i]}"
return True, None
```

2.3 Frontend — HMI (index.html)

The HMI is a single-file HTML/JS application served by Nginx. It provides complete operator control and real-time visibility into the production line:

- Start / Stop / Reset buttons — POST to Flask backend
- Status badge — IDLE / RUNNING / STOPPED with colour coding
- KPI cards — Produced, Defective, Good units, Yield %

- Error banner — flashes red with defect reason for 4 seconds on each fault
- Live Chart.js line charts — Good vs Defective units over time; Stage temperatures
- Production log table — last 10 units with status and defect reason
- Grafana link — direct link to the live dashboard

2.4 Database — InfluxDB 2.7

InfluxDB stores all production metrics in the **production** bucket under the **production_metrics** measurement. The following fields are written on every tick:

Field	Type	Description
produced	float	Cumulative units attempted
defective	float	Cumulative defective units
good	float	Cumulative good units
yield_pct	float	Good / Produced * 100
temp_stage1–4	float	Simulated temperature per stage (°C)
unit_ok	float	1.0 = pass, 0.0 = fail for this unit
event	str	start / stop / reset lifecycle events

Table 3 — InfluxDB fields written per production tick

2.5 Dashboard — Grafana 10.2

Grafana is auto-provisioned via the **grafana/provisioning/** directory included in the project. On first startup, Docker Compose mounts the provisioning folder and Grafana automatically creates the InfluxDB datasource (Flux query language) and loads the **pencil.json** dashboard without any manual configuration.

The provisioned dashboard displays the following panels:

- Total units produced — time-series line graph
- Good vs Defective units — stacked area chart
- Yield % — gauge panel with green/amber/red thresholds
- Stage temperatures (°C) — multi-line time-series for all 4 stages
- Machine state (running/stopped) — stat panel

Tools and Version Control

3.1 Git and GitHub

Version control was managed with **Git** throughout the project. The repository was hosted on GitHub, providing remote backup, commit history, and a branching strategy to isolate feature work from the main branch.

Key Git practices used:

- Initialised with git init and connected to a remote GitHub repository
- Frequent atomic commits with descriptive messages (e.g. 'feat: add InfluxDB write on tick')
- Feature branch pencil-production branched from main for development
- Merged via pull request after local testing to keep main stable
- GitHub Actions CI was considered to run a basic smoke test on the backend

3.2 Docker and Docker Compose

Docker was used to containerise every component of the stack, ensuring reproducible deployments regardless of the host operating system. **Docker Compose** orchestrates all four containers (backend, influxdb, grafana, frontend) with a single command:

```
docker compose up --build
```

Key Docker features used in this project:

- Custom Dockerfile for the Flask backend (Python 3.11-slim base image)
- Official images for InfluxDB 2.7 and Grafana 10.2 — no custom build needed
- Health check on InfluxDB container; backend uses depends_on with condition: service_healthy
- Named volumes (influx_data, grafana_data) for persistent storage across restarts
- Bind mount for Grafana provisioning folder — auto-configures datasource and dashboard
- Environment variables passed to backend for InfluxDB connection — no hardcoded secrets in code

3.3 AI Tools — Claude (Anthropic)

Claude by Anthropic was used as the primary AI coding assistant during this project. The following describes how it was used, the benefits observed, and corrections made.

How Claude was used:

- Generating the initial Flask REST API structure and InfluxDB integration boilerplate
- Designing the HMI layout with Chart.js for real-time visualisation
- Writing the docker-compose.yml with health checks and service dependencies
- Creating the Grafana provisioning YAML files for auto-configuration

Benefits:

- Significant reduction in boilerplate time — the InfluxDB write helper and Flask routes were generated in one prompt and required minimal adjustment
- Claude suggested using depends_on with service_healthy for InfluxDB, preventing a race condition where the backend started before the database was ready

- The Chart.js configuration for dual-dataset line charts was generated correctly on the first attempt, saving several hours of documentation reading

Errors found and corrections made:

- The initial AI-generated Grafana datasource YAML used the legacy InfluxQL query language instead of Flux, which is required for InfluxDB 2.x. This was corrected by changing jsonData.version from 'InfluxQL' to 'Flux'.
- The generated frontend initially called /tick on a setInterval inside the frontend JS but did not clear the interval on Stop — causing ghost ticks after stopping. An explicit clearInterval(tickTimer) was added to the stop and reset handlers.
- The InfluxDB Point write initially used integer fields; InfluxDB requires consistent types, so all numeric fields were cast to float() explicitly.

Conclusion

4.1 Summary of the Solution

The implemented pencil production line simulation successfully covers all four required components: a Python/Flask backend with realistic defect logic, a full-featured HMI frontend, an InfluxDB time-series database, and an auto-provisioned Grafana dashboard. The entire stack is containerised with Docker Compose and can be launched with a single command, making deployment straightforward and reproducible.

4.2 What Works Well

- The defect detection model is stage-specific — each stage has its own rate and a descriptive failure reason, making the simulation realistic and educational.
- The HMI provides immediate visual feedback: colour-coded KPIs, live charts, a scrolling log, and a flashing error banner make the machine state instantly readable.
- Grafana auto-provisioning eliminates all manual dashboard setup — the dashboard is ready immediately after docker compose up.
- The architecture is fully decoupled: replacing any single service (e.g. swapping InfluxDB for TimescaleDB) requires changes only to the relevant container and the backend write calls.

4.3 Weaknesses and Limitations

- The defect rates and temperature values are purely random — a more realistic simulation would model correlations between temperature and defect rate, or introduce gradual machine wear that increases fault probability over time.
- There is no user authentication on the HMI or Grafana (anonymous viewer access is enabled). In a real deployment, role-based access control would be mandatory.
- The production state is held in a Python dictionary in memory. A server restart loses all runtime state. Persisting state to InfluxDB or a relational database between sessions would be necessary for production use.
- The tick mechanism is driven by a JavaScript setInterval on the frontend. If the browser tab is inactive or throttled, ticks may be delayed or skipped, creating gaps in the InfluxDB time series.
- There is no alarm management system — defects are displayed and cleared after 4 seconds but are not stored in a dedicated alarm log with acknowledgement workflow.

4.4 Further Development

- Quality Control module: add a vision-inspection stage that captures a simulated image and uses an ML model to classify surface defects — this would fulfil the optional QC requirement from the project brief.
- CMMS integration: track machine uptime and maintenance intervals; trigger a simulated maintenance alert when the total defect count exceeds a threshold.
- SCADA layer: implement OPC-UA communication between the backend and a soft-PLC to simulate real industrial control protocols.
- Persistent alarm log: store all fault events in InfluxDB with a dedicated measurement and add a Grafana annotations panel to overlay faults on the time-series charts.
- Automated testing: add pytest unit tests for the defect logic and a Docker-based integration test that starts the full stack and verifies the /tick endpoint.

4.5 Operational Requirements

Deploying this solution in a real industrial environment would introduce several additional requirements. Network security would require VPN or firewall rules to isolate the production network. High availability would demand container orchestration (Kubernetes or Docker Swarm) with health-check-based auto-restart. Data retention policies in InfluxDB would need to be configured to manage storage growth over months of continuous operation. Finally, role-based access control in Grafana would separate operator, engineer, and manager views, ensuring that only authorised personnel can start or reset the line.

Appendix A — AI Prompt Log

The following prompts were submitted to Claude (Anthropic) during development:

- **Prompt 1:** "Create a Flask REST API for a pencil production line with 4 stages. Each stage should have a defect rate. Write results to InfluxDB 2.x using the influxdb-client Python library."
- **AI Output:** Generated the complete app.py including the influx_write helper, all five routes (/start, /stop, /reset, /tick, /status), and the STAGES / DEFECT_RATES / DEFECT_REASONS constants.
- **Correction:** All numeric fields cast to float() explicitly to avoid InfluxDB field type conflicts on repeated writes.
- **Prompt 2:** "Create a single-file HTML HMI with Start/Stop/Reset buttons, KPI cards for produced/defective/good/yield, a Chart.js line chart for units over time and one for stage temperatures, and a production log table."
- **AI Output:** Generated the complete index.html with all requested components.
- **Correction:** Added clearInterval(tickTimer) to the stop and reset handlers to prevent ghost ticks after the machine was stopped.
- **Prompt 3:** "Write a docker-compose.yml that runs Flask backend, InfluxDB 2.7, Grafana 10.2, and Nginx serving a static HTML file. Add a health check on InfluxDB so the backend waits for it to be ready."
- **AI Output:** Generated the full docker-compose.yml with health checks, volumes, and environment variables.
- **Correction:** Grafana datasource YAML corrected from InfluxQL to Flux query language for compatibility with InfluxDB 2.x.